JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time.temporal

Interface TemporalUnit

All Known Implementing Classes:

`ChronoUnit`

`public interface` **`TemporalUnit`**

A unit of date-time, such as Days or Hours.

Measurement of time is built on units, such as years, months, days, hours, minutes and seconds. Implementations of this interface represent those units.

An instance of this interface represents the unit itself, rather than an amount of the unit. See `Period` for a class that represents an amount in terms of the common units.

The most commonly used units are defined in `ChronoUnit`. Further units are supplied in `IsoFields`. Units can also be written by application code by implementing this interface.

The unit works using double dispatch. Client code calls methods on a date-time like `LocalDateTime` which check if the unit is a `ChronoUnit`. If it is, then the date-time must handle it. Otherwise, the method call is re-dispatched to the matching method in this interface.

Implementation Requirements:

`This interface must be implemented with care to ensure other classes operate correctly. All implementations that can be instantiated must be final, immutable and thread-safe. It is recommended to use an enum where possible.`

Since:

`1.8`

Method Summary

Default Methods **Abstract Methods**

Modifier and Type	Method	Description
<R extends Temporal> R	addTo(R temporal, long amount)	Returns a copy of the specified temporal object with the specified period added.
long	between(Temporal temporal1Inclusive, Temporal temporal2Exclusive)	Calculates the amount of time between two temporal objects.
Duration	getDuration()	Gets the duration of this unit, which may be an estimate.
boolean	isDateBased()	Checks if this unit represents a component of a date.
boolean	isDurationEstimated()	Checks if the duration of the unit is an estimate.
default boolean	isSupportedBy(Temporal temporal)	Checks if this unit is supported

		by the specified temporal object.
boolean	**isTimeBased**()	Checks if this unit represents a component of a time.
String	**toString**()	Gets a descriptive name for the unit.

Method Detail

getDuration

`Duration getDuration()`

Gets the duration of this unit, which may be an estimate.

All units return a duration measured in standard nanoseconds from this method. The duration will be positive and non-zero. For example, an hour has a duration of `60 * 60 * 1,000,000,000ns`.

Some units may return an accurate duration while others return an estimate. For example, days have an estimated duration due to the possibility of daylight
saving time changes. To determine if the duration is an estimate, use `isDurationEstimated()`.

Returns:

`the duration of this unit, which may be an estimate, not null`

isDurationEstimated

`boolean isDurationEstimated()`

Checks if the duration of the unit is an estimate.

All units have a duration, however the duration is not always accurate. For example, days have an estimated duration due to the possibility of daylight

saving time changes. This method returns true if the duration is an estimate and false if it is accurate. Note that accurate/estimated ignores leap seconds.

Returns:

`true if the duration is estimated, false if accurate`

isDateBased

```
boolean isDateBased()
```

Checks if this unit represents a component of a date.

A date is time-based if it can be used to imply meaning from a date. It must have a duration that is an integral multiple of the length of a standard day. Note that it is valid for both `isDateBased()` and `isTimeBased()` to return false, such as when representing a unit like 36 hours.

Returns:

`true if this unit is a component of a date`

isTimeBased

```
boolean isTimeBased()
```

Checks if this unit represents a component of a time.

A unit is time-based if it can be used to imply meaning from a time. It must have a duration that divides into the length of a standard day without remainder. Note that it is valid for both `isDateBased()` and `isTimeBased()` to return false, such as when representing a unit like 36 hours.

Returns:

`true if this unit is a component of a time`

isSupportedBy

```
default boolean isSupportedBy(Temporal temporal)
```

Checks if this unit is supported by the specified temporal object.

This checks that the implementing date-time can add/subtract this unit. This can be used to avoid throwing an exception.

This default implementation derives the value using `Temporal.plus(long, TemporalUnit)`.

Parameters:

temporal - the temporal object to check, not null

Returns:

true if the unit is supported

addTo

```
<R extends Temporal> R addTo(R temporal,
                             long amount)
```

Returns a copy of the specified temporal object with the specified period added.

The period added is a multiple of this unit. For example, this method could be used to add "3 days" to a date by calling this method on the instance representing "days", passing the date and the period "3". The period to be added may be negative, which is equivalent to subtraction.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `Temporal.plus(long, TemporalUnit)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisUnit.addTo(temporal);
temporal = temporal.plus(thisUnit);
```

It is recommended to use the second approach, `plus(TemporalUnit)`, as it is a lot clearer to read in code.

Implementations should perform any queries or calculations using the units available in `ChronoUnit` or the fields available in `ChronoField`. If the unit is not supported an `UnsupportedTemporalTypeException` must be thrown.

Implementations must not alter the specified temporal object. Instead, an adjusted copy of the original must be returned. This provides equivalent, safe behavior for immutable and mutable implementations.

Type Parameters:

R - the type of the Temporal object

Parameters:

temporal - the temporal object to adjust, not null

amount - the amount of this unit to add, positive or negative

Returns:

the adjusted temporal object, not null

Throws:

DateTimeException - if the amount cannot be added

UnsupportedTemporalTypeException - if the unit is not supported by the temporal

between

```
long between(Temporal temporal1Inclusive,
             Temporal temporal2Exclusive)
```

Calculates the amount of time between two temporal objects.

This calculates the amount in terms of this unit. The start and end points are supplied as temporal objects and must be of compatible types. The implementation will convert the second type to be an instance of the first type before the calculating the amount. The result will be negative if the end is before the start. For example, the amount in hours between two temporal objects can be calculated using `HOURS.between(startTime, endTime)`.

The calculation returns a whole number, representing the number of complete units between the two temporals. For example, the amount in hours between the times 11:30 and 13:29 will only be one hour as it is one minute short of two hours.

There are two equivalent ways of using this method. The first is to invoke this method directly. The second is to use `Temporal.until(Temporal, TemporalUnit)`:

```
// these two lines are equivalent
between = thisUnit.between(start, end);
between = start.until(end, thisUnit);
```

The choice should be made based on which makes the code more readable.

For example, this method allows the number of days between two dates to be calculated:

```
long daysBetween = DAYS.between(start, end);
// or alternatively
long daysBetween = start.until(end, DAYS);
```

Implementations should perform any queries or calculations using the units available in `ChronoUnit` or the fields available in `ChronoField`. If the unit is not supported an `UnsupportedTemporalTypeException` must be thrown. Implementations must not alter the specified temporal objects.

Implementation Requirements:

Implementations must begin by checking to if the two temporals have the same type using getClass(). If they do not, then the

result must be obtained by calling temporal1Inclusive.until(temporal2Exclusive, this).

Parameters:

temporal1Inclusive - the base temporal object, not null

temporal2Exclusive - the other temporal object, exclusive, not null

Returns:

the amount of time between temporal1Inclusive and temporal2Exclusive in terms of this unit; positive if temporal2Exclusive is later than temporal1Inclusive, negative if earlier

Throws:

DateTimeException - if the amount cannot be calculated, or the end temporal cannot be converted to the same type as the start temporal

UnsupportedTemporalTypeException - if the unit is not supported by the temporal

ArithmeticException - if numeric overflow occurs

toString

String toString()

Gets a descriptive name for the unit.

This should be in the plural and upper-first camel case, such as 'Days' or 'Minutes'.

Overrides:

toString in class Object

Returns:

```
the name of this unit, not null
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.